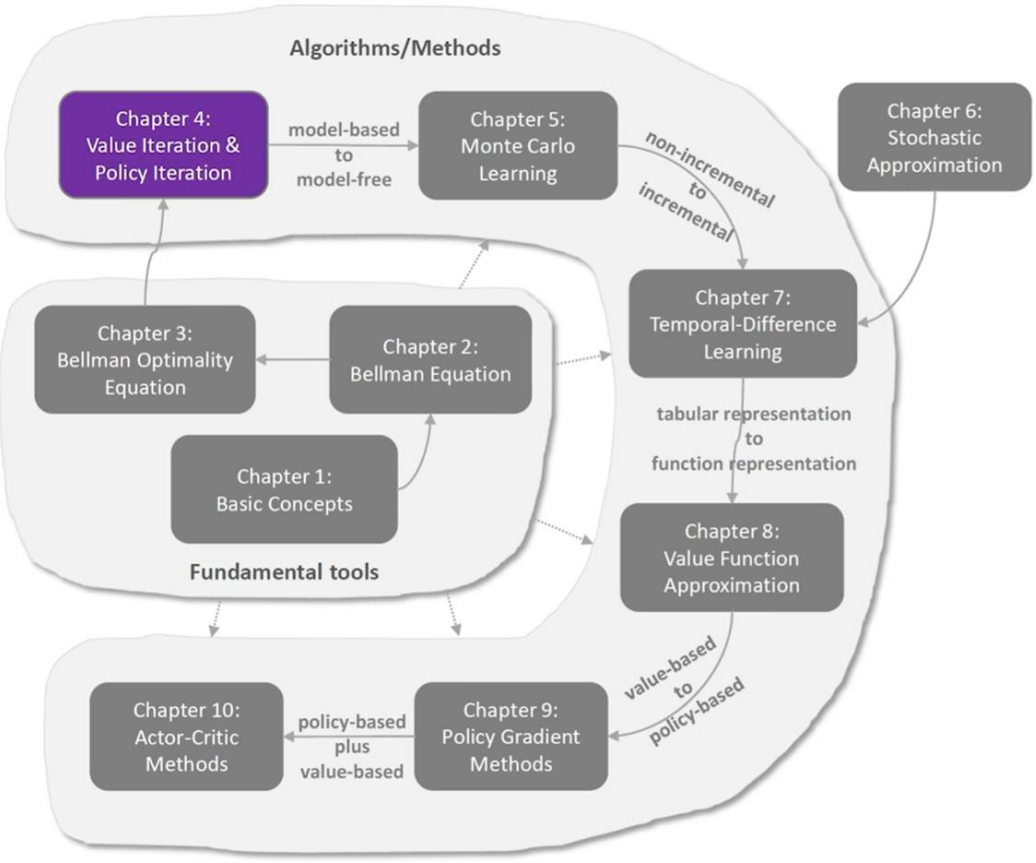
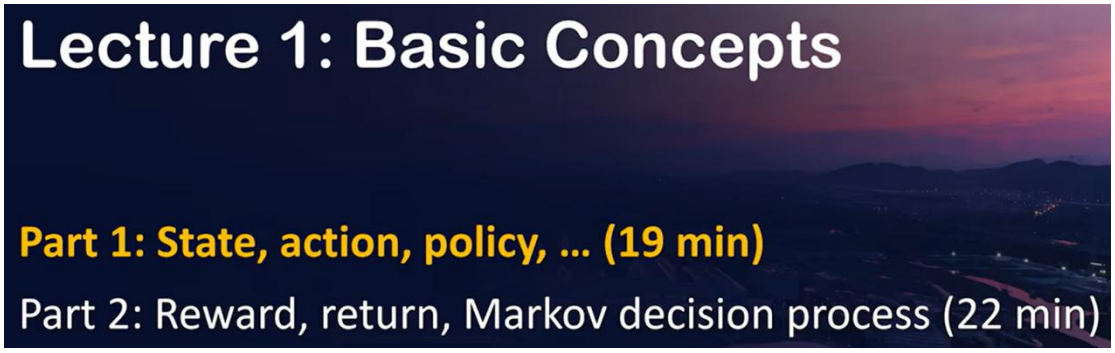
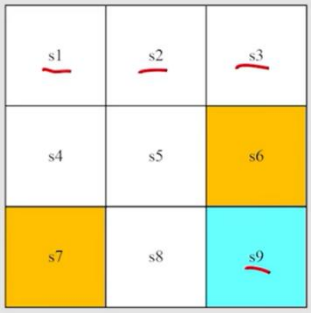
强化学习的数学原理


Algorithms/Methods
Chapter 4: Value Iteration & Policy Iteration
model-based to model-free
Chapter 5: Monte Carlo Learning
non-incremental to incremental
Chapter 6: Stochastic Approximation
Chapter 7: Temporal-Difference Learning
Chapter 3: Bellman Optimality Equation
Chapter 2: Bellman Equation
Chapter 1: Basic Concepts
Fundamental tools
tabular representation to function representation
Chapter 8: Value Function Approximation
value-based to policy-based
Chapter 10: Actor-Critic Methods
policy-based plus value-based
Chapter 9: Policy Gradient Methods


Lecture 1: Basic Concepts
Part 1: State, action, policy, … (19 min)
Part 2: Reward, return, Markov decision process (22 min)

State: The status of the agent with respect to the environment.

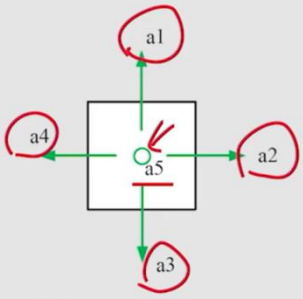
- For the grid-world example, the location of the agent is the state. There are nine possible locations and hence nine states: $s_1, s_2, \ldots, s_9$.

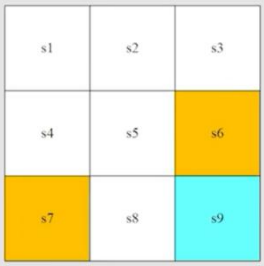


State space: the set of all states $\mathcal{S} = \{s_i\}_{i=1}^{9}$.

Action: For each state, there are five possible actions: $a_1, \ldots, a_5$

- a_1: move upwards;
- a_2: move rightwards;
- a_3: move downwards;
- a_4: move leftwards;
- a_5: stay unchanged;





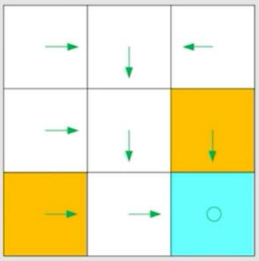
Action space of a state: the set of all possible actions of a state.
$\mathcal{A}(s_i) = \{a_i\}_{i=1}^{5}$.
Question: can different states have different sets of actions?

State transition

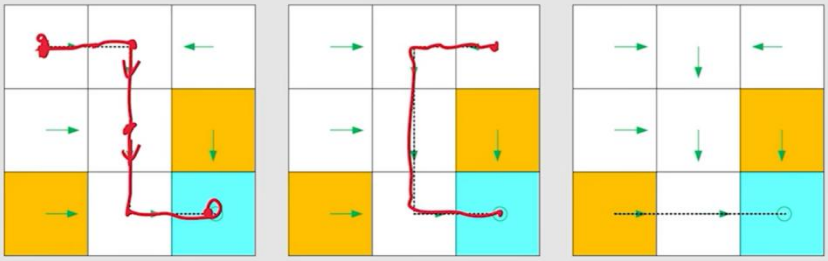
Policy

Policy tells the agent what actions to take at a state.

Intuitive representation: The arrows demonstrate a policy.



Based on this policy, we get the following paths with different starting points.



确定策略 vs 随机策略

Reward is one of the most unique concepts of RL.

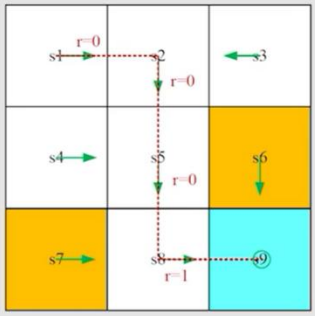
Reward: a real number we get after taking an action.

- A positive reward represents encouragement to take such actions.
- A negative reward represents punishment to take such actions.

Questions:

- What about a zero reward? No punishment.
- Can positive mean punishment? Yes.

Trajectory and return



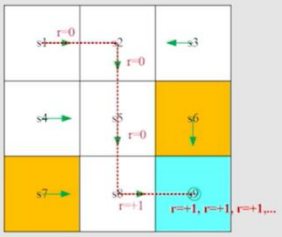
A *trajectory* is a state-action-reward chain:

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$$

The *return* of this trajectory is the sum of all the rewards collected along the trajectory:

$$\text{return} = 0 + 0 + 0 + 1 = 1$$

Discounted return



Need to introduce a *discount rate* $\gamma \in [0, 1)$

Discounted return:

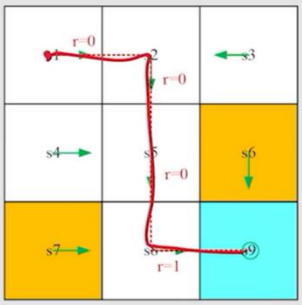
$$\begin{aligned}\text{discounted return} &= 0 + \gamma 0 + \gamma^2 0 + \gamma^3 1 + \gamma^4 1 + \gamma^5 1 + \dots \\ &= \gamma^3 (1 + \gamma + \gamma^2 + \dots) = \gamma^3 \frac{1}{1-\gamma}.\end{aligned}$$

Roles: 1) the sum becomes finite; 2) balance the far and near future rewards:

- If γ is close to 0, the value of the discounted return is dominated by the rewards obtained in the near future.
- If γ is close to 1, the value of the discounted return is dominated by the rewards obtained in the far future.

Episode

When interacting with the environment following a policy, the agent may stop at some *terminal states*. The resulting trajectory is called an *episode* (or a trial).



Example: episode

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$$

An episode is usually assumed to be a finite trajectory. Tasks with episodes are called *episodic tasks*.

MDP

Markov decision process (MDP)

个应该是a_t吗

Key elements of MDP:

- Sets:
  - State: the set of states $\mathcal{S}$
  - Action: the set of actions $\mathcal{A}(s)$ is associated for state $s \in \mathcal{S}$.
  - Reward: the set of rewards $\mathcal{R}(s, a)$.
- Probability distribution:
  - State transition probability: at state s, taking action a, the probability to transit to state s' is $p(s'|s, a)$
  - Reward probability: at state s, taking action a, the probability to get reward r is $p(r|s, a)$
- Policy: at state s, the probability to choose action a is $\pi(a|s)$
- *Markov property*: memoryless property

$$p(s_{t+1}|a_{t+1}, s_t, \ldots, a_1, s_0) = p(s_{t+1}|a_{t+1}, s_t),$$
$$p(r_{t+1}|a_{t+1}, s_t, \ldots, a_1, s_0) = p(r_{t+1}|a_{t+1}, s_t).$$

All the concepts introduced in this lecture can be put in the framework in MDP.

Lecture 2: Bellman Equation

Part 1: Motivating examples – why is return important? (16min)

Part 2: State value (7min)

Part 3: Bellman equation – Derivation (15 min)

Part 4: Bellman equation – Matrix-vector form and solution (13 min)

Part 5: Action value (10 min)

Return 判断策略好坏

计算 Return：1. 定义 2. 递归

Motivating example: How to calculate return?

How to solve these equations? Write in the following matrix-vector form:

$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{\mathbf{v}} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix} + \begin{bmatrix} \gamma v_2 \\ \gamma v_3 \\ \gamma v_4 \\ \gamma v_1 \end{bmatrix} = \underbrace{\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}}_{\mathbf{r}} + \gamma \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{\mathbf{P}} \underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{\mathbf{v}}$$

which can be rewritten as

$$\mathbf{v} = \mathbf{r} + \gamma \mathbf{P} \mathbf{v}$$

This is the Bellman equation (for this specific deterministic problem)!!

- Though simple, it demonstrates the core idea: the value of one state relies on the values of other states.
- A matrix-vector form is more clear to see how to solve the state values.

State Value

Some notations

Consider the following single-step process:

$$S_t \xrightarrow{A_t} R_{t+1}, S_{t+1}$$

- $t, t+1$: discrete time instances
- S_t: state at time t
- A_t: the action taken at state S_t
- R_{t+1}: the reward obtained after taking A_t
- S_{t+1}: the state transited to after taking A_t

Note that S_t, A_t, R_{t+1} are all *random variables*.

This step is governed by the following probability distributions:

- $S_t \to A_t$ is governed by $\pi(A_t = a|S_t = s)$
- $S_t, A_t \to R_{t+1}$ is governed by $p(R_{t+1} = r|S_t = s, A_t = a)$
- $S_t, A_t \to S_{t+1}$ is governed by $p(S_{t+1} = s'|S_t = s, A_t = a)$

At this moment, we assume we know the model (i.e., the probability distributions)!

State value

state value是广义的retrun

The expectation (or called expected value or mean) of G_t is defined as the *state-value function* or simply *state value*:

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s]$$

Remarks:

- It is a function of s. It is a conditional expectation with the condition that the state starts from s.
- It is based on the policy π. For a different policy, the state value may be different.
- It represents the "value" of a state. If the state value is greater, then the policy is better because greater cumulative rewards can be obtained.

Q: What is the relationship between return and state value?

A: The state value is the mean of all possible returns that can be obtained starting from a state. If everything - $\pi(a|s)$, $p(r|s,a)$, $p(s'|s,a)$ - is deterministic, then state value is the same as return.

贝尔曼方程的推导　PPT

Deriving the Bellman equation

Therefore, we have

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}[R_{t+1}|S_t = s] + \gamma \mathbb{E}[G_{t+1}|S_t = s], \\
&= \underbrace{\sum_a \pi(a|s) \sum_r p(r|s,a) r}_{\text{mean of immediate rewards}} + \gamma \underbrace{\sum_a \pi(a|s) \sum_{s'} p(s'|s,a) v_\pi(s')}_{\text{mean of future rewards}}, \\
&= \sum_a \pi(a|s) \left[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \right], \quad \forall s \in \mathcal{S}.
\end{aligned}
$$

Highlights:

- The above equation is called the *Bellman equation*, which characterizes the relationship among the state-value functions of different states.
- It consists of two terms: the immediate reward term and the future reward term.
- A set of equations: every state has an equation like this!!!

向量形式

Solve state values

Diagonal dominated 一定有逆　　用矩阵分解求解方便一点，比如LR分解

The Bellman equation in matrix-vector form is

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

- The *closed-form solution* is:

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

  In practice, we still need to use numerical tools to calculate the matrix inverse.

  Can we avoid the matrix inverse operation? Yes, by iterative algorithms.

- *An iterative solution is:*

$$v_{k+1} = r_\pi + \gamma P_\pi v_k$$

  This algorithm leads to a sequence $\{v_0, v_1, v_2, \dots\}$. We can show that

$$v_k \to v_\pi = (I - \gamma P_\pi)^{-1} r_\pi, \quad k \to \infty$$

Action value

From state value to action value:

- State value: the average return the agent can get *starting from a state*.
- Action value: the average return the agent can get *starting from a state* and *taking an action*.

Why do we care action value? Because we want to know which action is better. This point will be clearer in the following lectures.

We will frequently use action values.

Action value

不同的policy会导致路径不同，Gt和state va

Definition:

$$q_\pi(s,a) = \mathbb{E}[G_t|S_t = s, A_t = a]$$

- $q_\pi(s,a)$ is a function of the state-action pair (s,a)
- $q_\pi(s,a)$ depends on π

It follows from the properties of conditional expectation that

$$\underbrace{\mathbb{E}[G_t|S_t = s]}_{v_\pi(s)} = \sum_a \underbrace{\mathbb{E}[G_t|S_t = s, A_t = a]}_{q_\pi(s,a)} \pi(a|s)$$

Hence,

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a) \tag{2}$$

Action value

Recall that the state value is given by

$$v_\pi(s) = \sum_a \pi(a|s) \underbrace{\Big[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \Big]}_{q_\pi(s,a)} \tag{3}$$

By comparing (2) and (3), we have the **action-value function** as

$$q_\pi(s,a) = \sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \tag{4}$$

(2) and (4) are the two sides of the same coin:

- (2) shows how to obtain state values from action values.
- (4) shows how to obtain action values from state values.

Summary

Key concepts and results:

- State value: $v_\pi(s) = \mathbb{E}[G_t | S_t = s]$
- Action value: $q_\pi(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a]$
- The Bellman equation (elementwise form):

$$\begin{aligned} v_\pi(s) &= \sum_a \pi(a|s) \underbrace{\Big[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_\pi(s') \Big]}_{q_\pi(s,a)} \\ &= \sum_a \pi(a|s) q_\pi(s,a) \end{aligned}$$

- The Bellman equation (matrix-vector form):

$$v_\pi = r_\pi + \gamma P_\pi v_\pi$$

- How to solve the Bellman equation: closed-form solution, iterative solution

Lecture 3: Bellman Optimality Equation

Part 1: Motivating example – how to improve policies? (8 min)

Part 2: Optimal policy and Bellman optimality equation (11 min)

Part 3: More on the Bellman optimality equation (18 min)

Part 4: Interesting properties of optimal policies (14 min)

BOE

Optimal policy

The state value could be used to evaluate if a policy is good or not: if

$$v_{\pi_1}(s) \geq v_{\pi_2}(s) \quad \text{for all } s \in \mathcal{S}$$

then π_1 is "better" than π_2.

Definition

A policy π^* is optimal if $v_{\pi^*}(s) \geq v_{\pi}(s)$ for all s and for any other policy π.

The definition leads to many questions:

- Does the optimal policy exist?
- Is the optimal policy unique?
- Is the optimal policy stochastic or deterministic?
- How to obtain the optimal policy?

To answer these questions, we study the *Bellman optimality equation*.

Bellman optimality equation (elementwise form):

$$v(s) = \max_{\pi} \sum_{a} \pi(a|s) \left(\sum_{r} p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v(s') \right), \quad \forall s \in \mathcal{S}$$

$$= \max_{\pi} \sum_{a} \pi(a|s) q(s,a) \quad s \in \mathcal{S}$$

Remarks:

- $p(r|s,a), p(s'|s,a)$ are known.
- $v(s), v(s')$ are unknown and to be calculated.
- Is $\pi(s)$ known or unknown?

Bellman optimality equation (matrix-vector form):

$$v = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

where the elements corresponding to s or s' are

$$[r_\pi]_s \triangleq \sum_{a} \pi(a|s) \sum_{r} p(r|s,a) r,$$

$$[P_\pi]_{s,s'} = p(s'|s) \triangleq \sum_{a} \pi(a|s) \sum_{s'} p(s'|s,a)$$

Here $\max_\pi$ is performed elementwise.

Solve the Bellman optimality equation

The BOE is $v = \max_\pi(r_\pi + \gamma P_\pi v)$. Let

$$f(v) := \max_\pi(r_\pi + \gamma P_\pi v)$$

Then, the Bellman optimality equation becomes

$$v = f(v)$$

where

$$[f(v)]_s = \max_\pi \sum_a \pi(a|s) q(s,a), \quad s \in \mathcal{S}$$

Next, how to solve the equation?

Preliminaries: Contraction mapping theorem

Some concepts:

- Fixed point: $x \in X$ is a fixed point of $f : X \to X$ if

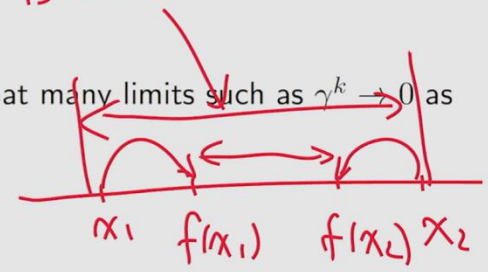
$$f(x) = x$$

- Contraction mapping (or contractive function): f is a contraction mapping if

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\|$$

where $\gamma \in (0, 1)$.

  - γ must be strictly less than 1 so that many limits such as $\gamma^k \to 0$ as $k \to 0$ hold.
  - Here $\|\cdot\|$ can be any vector norm.



Theorem (Contraction Mapping Theorem)

For any equation that has the form of $x = f(x)$, if f is a contraction mapping, then

- *Existence: there exists a fixed point x^* satisfying $f(x^*) = x^*$.*
- *Uniqueness: The fixed point x^* is unique.*
- *Algorithm: Consider a sequence $\{x_k\}$ where $x_{k+1} = f(x_k)$, then $x_k \to x^*$ as $k \to \infty$. Moreover, the convergence rate is exponentially fast.*

Applying the contraction mapping theorem gives the following results.

Theorem (Existence, Uniqueness, and Algorithm)

*For the BOE $v = f(v) = \max_\pi(r_\pi + \gamma P_\pi v)$, there always **exists** a solution v^* and the solution is **unique**. The solution could be solved iteratively by*

$$v_{k+1} = f(v_k) = \max_\pi(r_\pi + \gamma P_\pi v_k)$$

*This sequence $\{v_k\}$ converges to v^* **exponentially fast** given any initial guess v_0. The convergence rate is determined by γ.*

What does an optimal policy π^* look like?

Theorem (Greedy Optimal Policy)

For any $s \in \mathcal{S}$, the deterministic greedy policy

$$\pi^*(a|s) = \begin{cases} 1 & a = a^*(s) \\ 0 & a \neq a^*(s) \end{cases} \tag{1}$$

is an optimal policy solving the BOE. Here,

$$a^*(s) = \arg\max_a q^*(a, s),$$

where $q^*(s, a) := \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v^*(s')$.

Proof: simple. $\pi^*(s) = \arg\max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v^*(s') \right)}_{q^*(s,a)}$

Bellman optimality equation:

- Elementwise form:

$$v(s) = \max_{\pi} \sum_{a} \pi(a|s) \underbrace{\left(\sum_{r} p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v(s') \right)}_{q(s,a)}, \quad \forall s \in \mathcal{S}$$

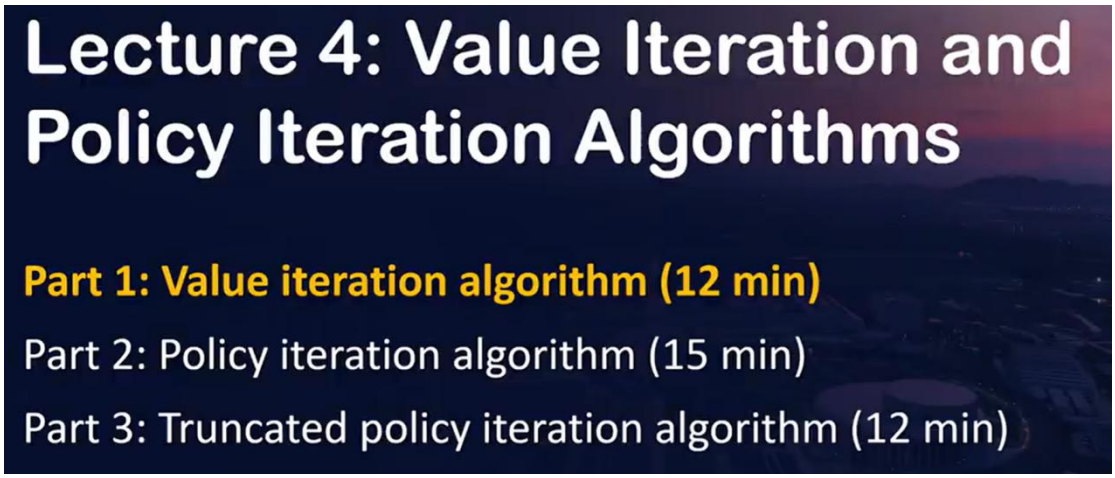
- Matrix-vector form:

$$v = \max_{\pi} (r_\pi + \gamma P_\pi v)$$

Questions about the Bellman optimality equation:

- Existence: does this equation have solutions?
  - Yes, by the contraction mapping Theorem
- Uniqueness: is the solution to this equation unique?
  - Yes, by the contraction mapping Theorem
- Algorithm: how to solve this equation?
  - Iterative algorithm suggested by the contraction mapping Theorem
- Optimality: why we study this equation
  - Because its solution corresponds to the optimal state value and optimal policy.

Finally, we understand why it is important to study the BOE!



动态规划

Value iteration algorithm

这里

▷ How to solve the Bellman optimality equation?

$$v = f(v) = \max_{\pi}(r_\pi + \gamma P_\pi v)$$

▷ In the last lecture, we know that the contraction mapping theorem suggest an iterative algorithm:

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 1, 2, 3 \ldots$$

where v_0 can be arbitrary.

▷ This algorithm can eventually find the optimal state value and an optimal policy.

▷ This algorithm is called *value iteration*!

▷ We will see that the math about the BOE that we have learned finally pays off!

Value iteration algorithm

主要还是因为

The algorithm

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 1, 2, 3 \ldots$$

can be decomposed to two steps.

- Step 1: policy update. This step is to solve

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k)$$

  where v_k is given.

- Step 2: value update.

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

Question: is v_k a state value? No, because it is not ensured that v_k satisfies a Bellman equation.

Value iteration algorithm - Pseudocode

▷ Procedure summary:

$v_k(s) \rightarrow q_k(s,a) \rightarrow$ greedy policy $\pi_{k+1}(a|s) \rightarrow$ new value $v_{k+1} = \max_a q_k(s,a)$

Pseudocode: Value iteration algorithm

Initialization: The probability model $p(r|s,a)$ and $p(s'|s,a)$ for all (s,a) are known. Initial guess v_0.

Aim: Search the optimal state value and an optimal policy solving the Bellman optimality equation.

While v_k has not converged in the sense that $\|v_k - v_{k-1}\|$ is greater than a predefined small threshold, for the kth iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

q-value: $q_k(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a)v_k(s')$

Maximum action value: $a_k^*(s) = \arg\max_a q_k(a,s)$

Policy update: $\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

Value update: $v_{k+1}(s) = \max_a q_k(a,s)$

Policy iteration algorithm

▷ Algorithm description:

Given a random initial policy π_0,

- Step 1: policy evaluation (PE)

  This step is to calculate the state value of π_k:

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

  Note that v_{π_k} is a state value function.

- Step 2: policy improvement (PI)

$$\pi_{k+1} = \arg\max_{\pi} (r_\pi + \gamma P_\pi v_{\pi_k})$$

  The maximization is componentwise!

Step 1: Policy evaluation

▷ Matrix-vector form: $v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0, 1, 2, \ldots$

▷ Elementwise form:

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right), \quad s \in \mathcal{S},$$

Stop when $j \to \infty$ or j is sufficiently large or $\|v_{\pi_k}^{(j+1)} - v_{\pi_k}^{(j)}\|$ is sufficiently small.

Step 2: Policy improvement

▷ Matrix-vector form: $\pi_{k+1} = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_k})$

▷ Elementwise form

$$\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, \quad s \in \mathcal{S}.$$

Here, $q_{\pi_k}(s,a)$ is the action value under policy π_k. Let

$$a_k^*(s) = \arg\max_a q_{\pi_k}(a,s)$$

Then, the greedy policy is

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s), \\ 0 & a \neq a_k^*(s). \end{cases}$$

Pseudocode: Policy iteration algorithm

Initialization: The probability model $p(r|s,a)$ and $p(s'|s,a)$ for all (s,a) are known. Initial guess π_0.

Aim: Search for the optimal state value and an optimal policy.

While the policy has not converged, for the kth iteration, do

Policy evaluation:

Initialization: an arbitrary initial guess $v_{\pi_k}^{(0)}$

While $v_{\pi_k}^{(j)}$ has not converged, for the jth iteration, do

For every state $s \in \mathcal{S}$, do

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s') \right]$$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$$q_{\pi_k}(s,a) = \sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')$$

$$a_k^*(s) = \arg\max_a q_{\pi_k}(s,a)$$

$$\pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise}$$

Compare value iteration and policy iteration

Policy iteration: start from π_0

- Policy evaluation (PE):

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$$

- Policy improvement (PI):

$$\pi_{k+1} = \arg\max_{\pi} (r_\pi + \gamma P_\pi v_{\pi_k})$$

Value iteration: start from v_0

- Policy update (PU):

$$\pi_{k+1} = \arg\max_{\pi} (r_\pi + \gamma P_\pi v_k)$$

- Value update (VU):

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k$$

▷ The two algorithms are very similar:

Policy iteration: $\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots$

Value iteration: $u_0 \xrightarrow{PU} \pi_1' \xrightarrow{VU} u_1 \xrightarrow{PU} \pi_2' \xrightarrow{VU} u_2 \xrightarrow{PU} \dots$

PE=policy evaluation. PI=policy improvement.

PU=policy update. VU=value update.

▷ Let's compare the steps carefully:

	Policy iteration algorithm	Value iteration algorithm	Comments
1) Policy:	π_0	N/A	
2) Value:	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 := v_{\pi_0}$	
3) Policy:	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_0})$	$\pi_1 = \arg\max_\pi (r_\pi + \gamma P_\pi v_0)$	The two policies are the same
4) Value:	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$	$v_{\pi_1} \geq v_1$ since $v_{\pi_1} \geq v_{\pi_0}$
5) Policy:	$\pi_2 = \arg\max_\pi (r_\pi + \gamma P_\pi v_{\pi_1})$	$\pi_2' = \arg\max_\pi (r_\pi + \gamma P_\pi v_1)$	
⋮	⋮	⋮	⋮

- They start from the same initial condition.
- The first three steps are the same.
- The fourth step becomes different:
  - In policy iteration, solving $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$ requires an iterative algorithm (an infinite number of iterations)
  - In value iteration, $v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$ is a one-step iteration

Consider the step of solving $v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$:

$$
\begin{aligned}
& & v_{\pi_1}^{(0)} &= v_0 \\
\text{value iteration} \leftarrow v_1 \longleftarrow & & v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \\
& & v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\
& & &\vdots \\
\text{truncated policy iteration} \leftarrow \bar{v}_1 \longleftarrow & & v_{\pi_1}^{(j)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \\
& & &\vdots \\
\text{policy iteration} \leftarrow v_{\pi_1} \longleftarrow & & v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)}
\end{aligned}
$$

- The value iteration algorithm computes *once*.
- The policy iteration algorithm computes *an infinite number of iterations*.
- The truncated policy iteration algorithm computes *a finite number of iterations* (say j). The rest iterations from j to ∞ are truncated.

Zhao

Pseudocode: Truncated policy iteration algorithm

Initialization: The probability model $p(r|s,a)$ and $p(s'|s,a)$ for all (s,a) are known. Initial guess π_0.

Aim: Search for the optimal state value and an optimal policy.

While the policy has not converged, for the kth iteration, do

Policy evaluation:

Initialization: select the initial guess as $v_k^{(0)} = v_{k-1}$. The maximum iteration is set to be j_{truncate}.

While $j < j_{\text{truncate}}$, do

For every state $s \in \mathcal{S}$, do

$$v_k^{(j+1)}(s) = \sum_a \pi_k(a|s) \left[\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_k^{(j)}(s') \right]$$

Set $v_k = v_k^{(j_{\text{truncate}})}$

Policy improvement:

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

$$q_k(s,a) = \sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_k(s')$$

$a_k^*(s) = \arg\max_a q_k(s,a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

▷ Value iteration: it is the iterative algorithm solving the Bellman optimality equation: given an initial value v_0,

$$v_{k+1} = \max_\pi(r_\pi + \gamma P_\pi v_k)$$

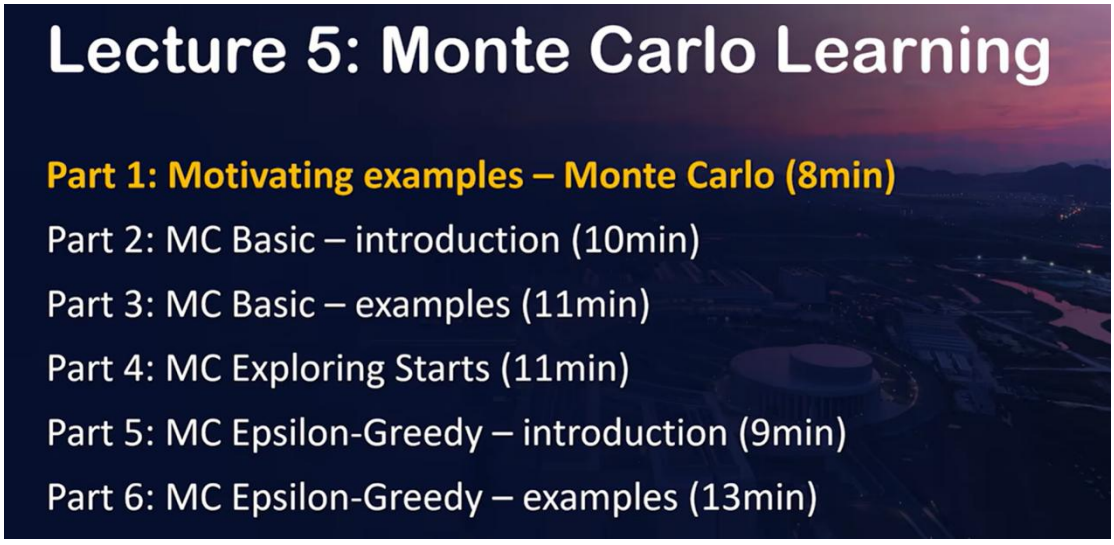
$$\Updownarrow$$

$$\begin{cases} \text{Policy update: } \pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_k) \\ \text{Value update: } v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \end{cases}$$

▷ Policy iteration: given an initial policy π_0,

$$\begin{cases} \text{Policy evaluation: } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{Policy improvement: } \pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_{\pi_k}) \end{cases}$$

▷ Truncated policy iteration



Model-free

大数定律

MC Basic

Law of Large Numbers

For a random variable X. Suppose $\{x_j\}_{j=1}^N$ are some iid samples. Let $\bar{x} = \frac{1}{N}\sum_{j=1}^N x_j$ be the average of the samples. Then,

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X],$$
$$\mathrm{Var}[\bar{x}] = \frac{1}{N}\mathrm{Var}[X].$$

As a result, $\bar{x}$ is an unbiased estimate of $\mathbb{E}[X]$ and its variance decreases to zero as N increases to infinity.

▷ The samples must be iid (independent and identically distributed)

▷ For the proof, see the book.

▷ Summary:

- Monte Carlo estimation refers to a broad class of techniques that rely on repeated random sampling to solve approximation problems.
- Why we care about Monte Carlo estimation? Because it does not require the model!
- Why we care about mean estimation? Because state value and action value are defined as expectations of random variables!

The procedure of Monte Carlo estimation of action values:

- Starting from (s, a), following policy π_k, generate an episode.
- The return of this episode is $g(s, a)$
- $g(s, a)$ is a sample of G_t in

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- Suppose we have a set of episodes and hence $\{g^{(j)}(s, a)\}$. Then,

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^{N} g^{(i)}(s, a).$$

Fundamental idea: When model is unavailable, we can use data.

▷ Description of the algorithm:

Given an initial policy π_0, there are two steps at the kth iteration.

- **Step 1: policy evaluation.** This step is to obtain $q_{\pi_k}(s, a)$ for all (s, a). Specifically, for each action-state pair (s, a), run an infinite number of (or sufficiently many) episodes. The average of their returns is used to approximate $q_{\pi_k}(s, a)$.
- **Step 2: policy improvement.** This step is to solve $\pi_{k+1}(s) = \arg\max_\pi \sum_a \pi(a|s) q_{\pi_k}(s, a)$ for all $s \in \mathcal{S}$. The greedy optimal policy is $\pi_{k+1}(a_k^*|s) = 1$ where $a_k^* = \arg\max_a q_{\pi_k}(s, a)$.

Exactly the same as the policy iteration algorithm, except

- Estimate $q_{\pi_k}(s, a)$ directly, instead of solving $v_{\pi_k}(s)$.

Pseudocode: MC Basic algorithm (a model-free variant of policy iteration)

Initialization: Initial guess π_0.

Aim: Search for an optimal policy.

While the value estimate has not converged, for the kth iteration, do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

Collect sufficiently many episodes starting from (s, a) following π_k

MC-based policy evaluation step:

$q_{\pi_k}(s, a) =$ average return of all the episodes starting from (s, a)

Policy improvement step:

$a_k^*(s) = \arg\max_a q_{\pi_k}(s, a)$

$\pi_{k+1}(a|s) = 1$ if $a = a_k^*$, and $\pi_{k+1}(a|s) = 0$ otherwise

MC Exploring Starts

Pseudocode: MC Exploring Starts (a sample-efficient variant of MC Basic)

Initialization: Initial guess π_0.

Aim: Search for an optimal policy.

For each episode, do

Episode generation: Randomly select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected. Following the current policy, generate an episode of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.

Policy evaluation and policy improvement:

Initialization: $g \leftarrow 0$

For each step of the episode, $t = T-1, T-2, \ldots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

Use the first-visit strategy:

If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1})$, then

$Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$

$q(s_t, a_t) =$ average$(Returns(s_t, a_t))$

$\pi(a|s_t) = 1$ if $a = \arg\max_a q(s_t, a)$

MC Epsilon-Greedy

Pseudocode: MC Exploring Starts (a sample-efficient variant of MC Basic)

Initialization: Initial guess π_0.
Aim: Search for an optimal policy.

For each episode, do

Episode generation: Randomly select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected. Following the current policy, generate an episode of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.

Policy evaluation and policy improvement:

Initialization: $g \leftarrow 0$

For each step of the episode, $t = T-1, T-2, \ldots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

Use the first-visit strategy:

If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1})$, then

$Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$

$q(s_t, a_t) = \text{average}(Returns(s_t, a_t))$

$\pi(a|s_t) = 1$ if $a = \arg\max_a q(s_t, a)$

▷ **How to embed ε-greedy into the MC-based RL algorithms?**

Originally, the policy improvement step in MC Basic and MC Exploring Starts is to solve

$$\pi_{k+1}(s) = \arg\max_{\pi \in \Pi} \sum_a \pi(a|s) q_{\pi_k}(s, a).$$

where Π denotes the set of all possible policies. The optimal policy here is

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*, \\ 0, & a \neq a_k^*, \end{cases}$$

where $a_k^* = \arg\max_a q_{\pi_k}(s, a)$.

▷ **How to embed ε-greedy into the MC-based RL algorithms?**

Now, the policy improvement step is changed to solve

$$\pi_{k+1}(s) = \arg\max_{\pi \in \Pi_\varepsilon} \sum_a \pi(a|s) q_{\pi_k}(s,a),$$

where Π_ε denotes the set of all ε-greedy policies with a fixed value of ε.

The optimal policy here is

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|}\varepsilon, & a = a_k^*, \\ \frac{1}{|\mathcal{A}(s)|}\varepsilon, & a \neq a_k^*. \end{cases}$$

- MC ε-Greedy is the *same* as that of MC Exploring Starts *except* that the former uses ε-greedy policies.
- It does not require exploring starts, but still requires to visit all state-action pairs in a different form.

Pseudocode: MC ϵ-Greedy (a variant of MC Exploring Starts)

Initialization: Initial guess π_0 and the value of $\epsilon \in [0,1]$

Aim: Search for an optimal policy.

For each episode, do

Episode generation: Randomly select a starting state-action pair (s_0, a_0). Following the current policy, generate an episode of length T: $s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T$.

Policy evaluation and policy improvement:

Initialization: $g \leftarrow 0$

For each step of the episode, $t = T-1, T-2, \ldots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

Use the every-visit method:

If (s_t, a_t) does not appear in $(s_0, a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1})$, then

$Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$

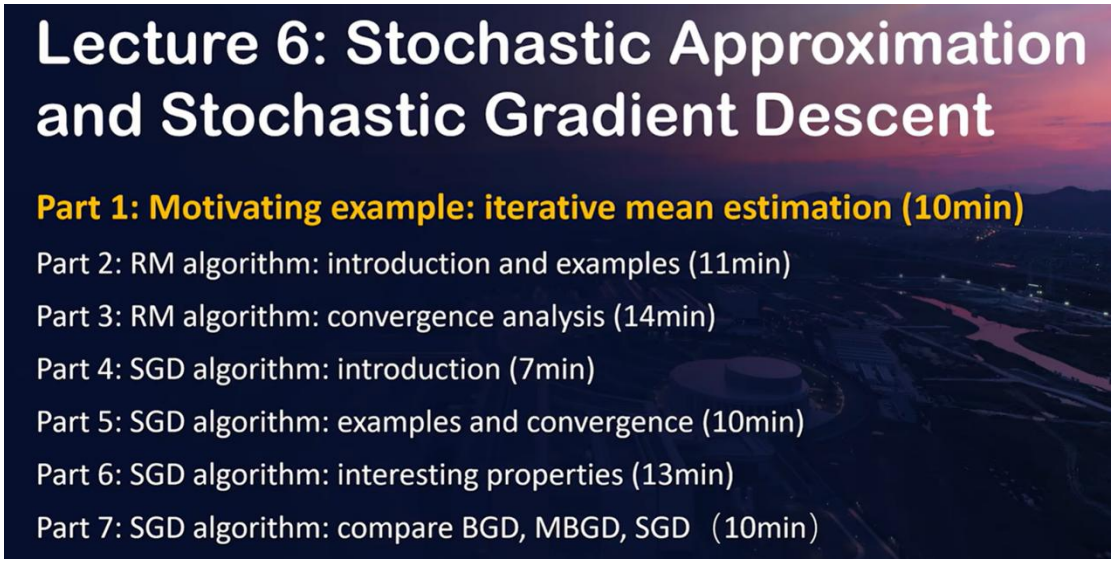
$q(s_t, a_t) =$ average$(Returns(s_t, a_t))$

Let $a^* = \arg\max_a q(s_t, a)$ and

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$$

Key points:

- Mean estimation by the Monte Carlo methods
- Three algorithms:
  - MC Basic
  - MC Exploring Starts
  - MC ε-Greedy
- Relationship among the three algorithms
- Optimality vs exploration of ε-greedy policies



有点迷

Motivating example: mean estimation

In particular, suppose

$$w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i, \quad k = 1, 2, \dots$$

and hence

$$w_k = \frac{1}{k-1}\sum_{i=1}^{k-1} x_i, \quad k = 2, 3, \dots$$

Then, w_{k+1} can be expressed in terms of w_k as

$$\begin{aligned} w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i &= \frac{1}{k}\left(\sum_{i=1}^{k-1} x_i + x_k\right) \\ &= \frac{1}{k}((k-1)w_k + x_k) = w_k - \frac{1}{k}(w_k - x_k). \end{aligned}$$

Therefore, we obtain the following iterative algorithm:

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

Robbins-Monro algorithm

Stochastic approximation (SA):

- SA refers to a broad class of stochastic iterative algorithms solving root finding or optimization problems.
- Compared to many other root-finding algorithms such as gradient-based methods, SA is powerful in the sense that it does *not* require to know the expression of the objective function nor its derivative.

Robbins-Monro (RM) algorithm:

- The is a pioneering work in the field of stochastic approximation.
- The famous stochastic gradient descent algorithm is a special form of the RM algorithm.
- It can be used to analyze the mean estimation algorithms introduced in the beginning.

Robbins-Monro algorithm – The algorithm

The Robbins-Monro (RM) algorithm can solve this problem:

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k), \qquad k = 1, 2, 3, \dots$$

where

- w_k is the kth estimate of the root
- $\tilde{g}(w_k, \eta_k) = g(w_k) + \eta_k$ is the kth noisy observation
- a_k is a positive coefficient.

The function $g(w)$ is a black box! This algorithm relies on data:

- Input sequence: $\{w_k\}$
- Noisy output sequence: $\{\tilde{g}(w_k, \eta_k)\}$

Philosophy: without model, we need data!

- Here, the model refers to the expression of the function.

Stochastic gradient descent

Method 1: gradient descent (GD)

$$w_{k+1} = w_k - \alpha_k \nabla_w \mathbb{E}[f(w_k, X)] = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)]$$

Drawback: the expected value is difficult to obtain.

Method 2: batch gradient descent (BGD)

$$\mathbb{E}[\nabla_w f(w_k, X)] \approx \frac{1}{n} \sum_{i=1}^{n} \nabla_w f(w_k, x_i).$$

$$w_{k+1} = w_k - \alpha_k \frac{1}{n} \sum_{i=1}^{n} \nabla_w f(w_k, x_i).$$

Drawback: it requires many samples in each iteration for each w_k.

Method 3: stochastic gradient descent (SGD)

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k),$$

- Compared to the gradient descent method: Replace the true gradient $\mathbb{E}[\nabla_w f(w_k, X)]$ by the stochastic gradient $\nabla_w f(w_k, x_k)$.
- Compared to the batch gradient descent method: let $n = 1$.

BGD, MBGD, and SGD

Suppose we would like to minimize $J(w) = \mathbb{E}[f(w, X)]$ given a set of random samples $\{x_i\}_{i=1}^n$ of X. The BGD, SGD, MBGD algorithms solving this problem are, respectively,

$$w_{k+1} = w_k - \alpha_k \frac{1}{n} \sum_{i=1}^{n} \nabla_w f(w_k, x_i), \qquad \text{(BGD)}$$

$$w_{k+1} = w_k - \alpha_k \frac{1}{m} \sum_{j \in \mathcal{I}_k} \nabla_w f(w_k, x_j), \qquad \text{(MBGD)}$$

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k). \qquad \text{(SGD)}$$

- **In the BGD algorithm**, all the samples are used in every iteration. When n is large, $(1/n)\sum_{i=1}^n \nabla_w f(w_k, x_i)$ is close to the true gradient $\mathbb{E}[\nabla_w f(w_k, X)]$.
- **In the MBGD algorithm**, $\mathcal{I}_k$ is a subset of $\{1, \ldots, n\}$ with the size as $|\mathcal{I}_k| = m$. The set $\mathcal{I}_k$ is obtained by m times idd samplings.
- **In the SGD algorithm**, x_k is randomly sampled from $\{x_i\}_{i=1}^n$ at time k.

Summary

- Mean estimation: compute $\mathbb{E}[X]$ using $\{x_k\}$

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k).$$

- RM algorithm: solve $g(w) = 0$ using $\{\tilde{g}(w_k, \eta_k)\}$

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k)$$

- SGD algorithm: minimize $J(w) = \mathbb{E}[f(w, X)]$ using $\{\nabla_w f(w_k, x_k)\}$

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k),$$

These results are useful:

- We will see in the next chapter that the temporal-difference learning algorithms can be viewed as stochastic approximation algorithms and hence have similar expressions.
- They are important optimization techniques that can be applied to many other fields.

Lecture 7: Temporal-Difference Learning

Part 1: Motivating example (9min)

Part 2: TD algorithm: introduction and interpretation (14min)

Part 3: TD algorithm: convergence and comparison (16min)

Part 4: Sarsa (16min)

Part 5: Expected Sarsa and n-step Sarsa (14min)

Part 6: Q-learning: introduction and on-policy vs off-policy (17min)

Part 7: Q-learning: pseudo code and examples (8min)

Part 8: Unified viewpoint and summary (10min)

Motivating example: stochastic algorithms

倒过来看你就明白了

We next consider some stochastic problems and show how to use the RM algorithm to solve them.

First, consider the simple mean estimation problem: calculate

$$w = \mathbb{E}[X],$$

based on some iid samples $\{x\}$ of X.

- By writing $g(w) = w - \mathbb{E}[X]$, we can reformulate the problem to a root-finding problem

$$g(w) = 0.$$

- Since we can only obtain samples $\{x\}$ of X, the noisy observation is

$$\tilde{g}(w, \eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta.$$

- Then, according to the last lecture, we know the RM algorithm for solving $g(w) = 0$ is

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k(w_k - x_k)$$

什么意思，哈哈哈

TD learning of state values – Algorithm description



The data/experience required by the algorithm:

- $(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots)$ or $\{(s_t, r_{t+1}, s_{t+1})\}_t$ generated following the given policy π.

$V_\pi(s)$

$V_t(s_t)$ state space

The TD learning algorithm is

$$\begin{cases} v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\left[v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]\right], & (1) \\ v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t, & (2) \end{cases}$$

where $t = 0, 1, 2, \ldots$. Here, $v_t(s_t)$ is the estimated state value of $v_\pi(s_t)$; $\alpha_t(s_t)$ is the learning rate of s_t at time t.

- At time t, only the value of the visited state s_t is updated whereas the values of the unvisited states $s \neq s_t$ remain unchanged.
- The update in (2) will be omitted when the context is clear.

The TD algorithm can be annotated as

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t)\Big[\overbrace{v_t(s_t) - \underbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}_{\text{TD target } \bar{v}_t}}^{\text{TD error } \delta_t}\Big], \quad (3)$$

Here,

$$\bar{v}_t \doteq r_{t+1} + \gamma v(s_{t+1})$$

is called the TD target.

$$\delta_t \doteq v(s_t) - [r_{t+1} + \gamma v(s_{t+1})] = v(s_t) - \bar{v}_t$$

is called the TD error.

It is clear that the new estimate $v_{t+1}(s_t)$ is a combination of the current estimate $v_t(s_t)$ and the TD error.

While TD learning and MC learning are both model-free, what are the **advantages and disadvantages** of TD learning compared to MC learning?

TD/Sarsa learning	MC learning
Online: TD learning is online. It can update the state/action values immediately after receiving a reward.	**Offline:** MC learning is offline. It has to wait until an episode has been completely collected.
Continuing tasks: Since TD learning is online, it can handle both episodic and continuing tasks.	**Episodic tasks:** Since MC learning is offline, it can only handle episodic tasks that has terminate states.

Table: Comparison between TD learning and MC learning.

While TD learning and MC learning are both model-free, what are the **advantages and disadvantages** of TD learning compared to MC learning?

TD/Sarsa learning	MC learning
Bootstrapping: TD bootstraps because the update of a value relies on the previous estimate of this value. Hence, it requires initial guesses.	**Non-bootstrapping:** MC is not bootstrapping, because it can directly estimate state/action values without any initial guess.
Low estimation variance: TD has lower than MC because there are fewer random variables. For instance, Sarsa requires $R_{t+1}, S_{t+1}, A_{t+1}$.	**High estimation variance:** To estimate $q_\pi(s_t, a_t)$, we need samples of $R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$. Suppose the length of each episode is L. There are $\lvert\mathcal{A}\rvert^L$ possible episodes.

Table: Comparison between TD learning and MC learning (continued).

Sarsa – Algorithm

我觉得这个地方用s'和a'代替t+1更好

First, our aim is to estimate the action values of a given policy π.

Suppose we have some experience $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$.

We can use the following *Sarsa* algorithm to estimate the action values:

$$\begin{cases} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t), \end{cases}$$

where $t = 0, 1, 2, \ldots$.

- $q_t(s_t, a_t)$ is an estimate of $q_\pi(s_t, a_t)$;
- $\alpha_t(s_t, a_t)$ is the learning rate depending on s_t, a_t.

The Q-learning algorithm is

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)] \right],$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

Q-learning is very similar to Sarsa. They are different only in terms of the TD target:

- The TD target in Q-learning is $r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)$
- The TD target in Sarsa is $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$.

Before further studying Q-learning, we first introduce two important concepts: on-policy learning and off-policy learning.

There exist two policies in a TD learning task:

- The behavior policy is used to generate experience samples.
- The target policy is constantly updated toward an optimal policy.

On-policy vs off-policy:

- When the behavior policy is the same as the target policy, such kind of learning is called on-policy.
- When they are different, the learning is called off-policy.

Sarsa 是 on-policy

蒙特卡洛是 on-policy

Q-learning 是 off-policy

Pseudocode: Policy searching by Q-learning (on-policy version)

For each episode, do

If the current s_t is not the target state, do

Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}.

Update q-value:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)]\Big]$$

Update policy:

$$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}|}(|\mathcal{A}| - 1) \text{ if } a = \arg\max_a q_{t+1}(s_t, a)$$

$$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}|} \text{ otherwise}$$

TD 算法的统一形式

A unified point of view

All the algorithms we introduced in this lecture can be expressed in a unified expression:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t],$$

where $\bar{q}_t$ is the *TD target*.

Different TD algorithms have different $\bar{q}_t$.

Algorithm	**Expression of $\bar{q}_t$**
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n-step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Expected Sarsa	$\bar{q}_t = r_{t+1} + \gamma \sum_a \pi_t(a\|s_{t+1}) q_t(s_{t+1}, a)$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Monte Carlo	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \ldots$

The MC method can also be expressed in this unified expression by setting $\alpha_t(s_t, a_t) = 1$ and hence $q_{t+1}(s_t, a_t) = \bar{q}_t$.

All the algorithms can be viewed as stochastic approximation algorithms solving the Bellman equation or Bellman optimality equation:

Algorithm	Equation aimed to solve
Sarsa	BE: $q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \middle| S_t = s, A_t = a\right]$
n-step Sarsa	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(s_{t+n}, a_{t+n}) \middle| S_t = s, A_t = a]$
Expected Sarsa	BE: $q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \mathbb{E}_{A_{t+1}}[q_\pi(S_{t+1}, A_{t+1})] \middle| S_t = s, A_t = a\right]$
Q-learning	BOE: $q(s,a) = \mathbb{E}\left[R_{t+1} + \max_a q(S_{t+1}, a) \middle| S_t = s, A_t = a\right]$
Monte Carlo	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \ldots \middle| S_t = s, A_t = a]$

Lecture 8: Value Function Approximation

Part 1: Motivating example – curve fitting (14min)

Part 2: Foundation – objective function (13min)

Part 3: Foundation – optimization algorithm and function selection (13min)

Part 4: Foundation – illustrative examples and analysis (13min)

Part 5: Sarsa and Q-learning (8min)

Part 6: Deep Q-learning – basic idea (10min)

Part 7: Deep Q-learning – experience replay (10min)

Part 8: Deep Q-learning – implementation and example (14min)

Up to now, we finished the story of TD learning with value function approximation.

1) This story started from the objective function:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2].$$

The objective function suggests that it is a policy evaluation problem.

2) The gradient-descent algorithm is

$$w_{t+1} = w_t + \alpha_t(v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t),$$

3) The true value function, which is unknown, in the algorithm is replaced by an approximation, leading to the algorithm:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right] \nabla_w \hat{v}(s_t, w_t).$$

Although this story is helpful to understand the basic idea, it is not mathematically rigorous.

So far, we merely considered the problem of state value estimation. That is we hope

$$\hat{v} \approx v_\pi$$

To search for optimal policies, we need to estimate action values.

The Sarsa algorithm with value function approximation is

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)\right] \nabla_w \hat{q}(s_t, a_t, w_t).$$

This is the same as the algorithm we introduced previously in this lecture except that $\hat{v}$ is replaced by $\hat{q}$.

Pseudocode: Sarsa with function approximation

Aim: Search a policy that can lead the agent to the target from an initial state-action pair (s_0, a_0).

For each episode, do

If the current s_t is not the target state, do

Take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}, and then take action a_{t+1} following $\pi_t(s_{t+1})$

Value update (parameter update):

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Policy update:

$$\pi_{t+1}(a|s_t) = 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) \quad \text{if } a = \arg\max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$$

$$\pi_{t+1}(a|s_t) = \frac{\varepsilon}{|\mathcal{A}(s)|} \quad \text{otherwise}$$

Similar to Sarsa, tabular Q-learning can also be extended to the case of value function approximation.

The q-value update rule is

$$w_{t+1} = w_t + \alpha_t \Big[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \Big] \nabla_w \hat{q}(s_t, a_t, w_t),$$

which is the same as Sarsa except that $\hat{q}(s_{t+1}, a_{t+1}, w_t)$ is replaced by $\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t)$.

Pseudocode: Q-learning with function approximation (on-policy version)

Initialization: Initial parameter vector w_0. Initial policy π_0. Small $\varepsilon > 0$.

Aim: Search a good policy that can lead the agent to the target from an initial state-action pair (s_0, a_0).

For each episode, do

If the current s_t is not the target state, do

Take action a_t following $\pi_t(s_t)$, and generate r_{t+1}, s_{t+1}

Value update (parameter update):

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Policy update:

$$\pi_{t+1}(a|s_t) = 1 - \frac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)| - 1) \quad \text{if} \quad a = \arg\max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a, w_{t+1})$$

$$\pi_{t+1}(a|s_t) = \frac{\varepsilon}{|\mathcal{A}(s)|} \quad \text{otherwise}$$

Deep Q-learning aims to minimize the objective function/loss function:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right],$$

where (S, A, R, S') are random variables.

- This is actually the Bellman optimality error. That is because

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) \middle| S_t = s, A_t = a \right], \quad \forall s, a$$

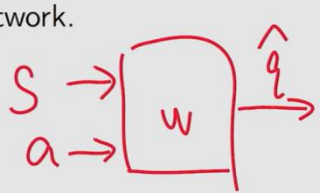
The value of $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)$ should be zero in the expectation sense

Deep Q-learning or deep Q-network (DQN):

- One of the earliest and most successful algorithms that introduce deep neural networks into RL.
- The role of neural networks is to be a nonlinear function approximator.
- Different from the following algorithm:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t),$$

because of the way of training a network.



Deep Q-learning aims to minimize the objective function/loss function:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right],$$

where (S, A, R, S') are random variables.

- This is actually the Bellman optimality error. That is because

$$q(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) \middle| S_t = s, A_t = a \right], \quad \forall s, a$$

The value of $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)$ should be zero in the expectation sense

To do that, we can introduce two networks.

- One is a main network representing $\hat{q}(s, a, w)$
- The other is a target network $\hat{q}(s, a, w_T)$.

The objective function in this case degenerates to

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right],$$

where w_T is the target network parameter.



Experience reply

Revisit the tabular case:

- Question: Why does not tabular Q-learning require experience replay?
  - Answer: No uniform distribution requirement.
- Question: Why Deep Q-learning involves distribution?
  - Answer: The objective function in the deep case is a *scalar* average over all (S, A). The tabular case does not involve any distribution of S or A. The algorithm in the tabular case aims to solve a set of equations for all (s, a) (Bellman optimality equation).
- Question: Can we use experience replay in tabular Q-learning?
  - Answer: Yes, we can. And more sample efficient (why?)

Deep Q-learning

Pseudocode: Deep Q-learning (off-policy version)

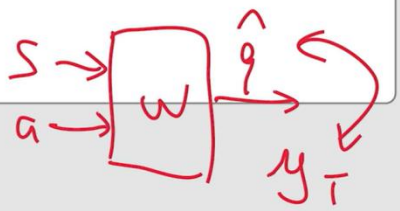
Aim: Learn an optimal target network to approximate the optimal action values from the experience samples generated by a behavior policy π_b.

Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$

For each iteration, do

- Uniformly draw a mini-batch of samples from $\mathcal{B}$
- For each sample (s, a, r, s'), calculate the target value as $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a, w_T)$, where w_T is the parameter of the target network
- Update the main network to minimize $(y_T - \hat{q}(s, a, w))^2$ using the mini-batch $\{(s, a, y_T)\}$

Set $w_T = w$ every C iterations



Remarks:

- Why no policy update?
- Why not using the policy update equation that we derived?
- The network input and output are different from the DQN paper.

Lecture 9: Policy Gradient Methods

Part 1: Basic idea (10min)

Part 2: Metric 1 – Average value (9min)

Part 3: Metric 2 – Average reward (11min)

Part 4: Gradients of the metrics (11min)

Part 5: Gradient-based algorithms and REINFORCE (16min)

REINFORCE

Pseudocode: Policy Gradient by Monte Carlo (REINFORCE)

Initialization: A parameterized function $\pi(a|s,\theta)$, $\gamma \in (0,1)$, and $\alpha > 0$.

Aim: Search for an optimal policy maximizing $J(\theta)$.

For the kth iteration, do

  Select s_0 and generate an episode following $\pi(\theta_k)$. Suppose the episode is $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$.

  For $t = 0, 1, \ldots, T-1$, do

    Value update: $q_t(s_t, a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$

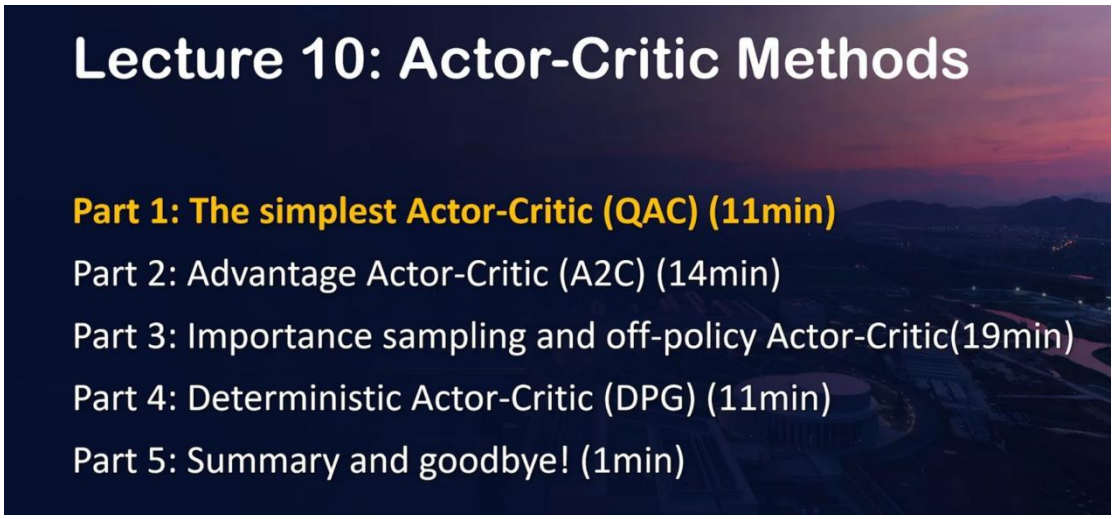
    Policy update: $\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t, a_t)$

  $\theta_k = \theta_T$

局限性与下一步（Actor-Critic）：

REINFORCE 使用蒙特卡洛求 G_t，方差太大。我们能不能结合 Lecture 7 的 TD 学习，用另一个神经网络去估计 Q 或 V 来代替 G_t 呢?

这就是强化学习另一座高峰——**Actor-Critic 方法**（演员-评论家算法），也是现代深度强化学习（如 A3C, PPO, SAC）的基础。



Actor-critic methods are still policy gradient methods.

- They emphasize the structure that incorporates the policy gradient and value-based methods.

What are "actor" and "critic"?

- Here, "actor" refers to policy update. It is called *actor* is because the policies will be applied to take actions.
- Here, "critic" refers to policy evaluation or value estimation. It is called *critic* because it criticizes the policy by evaluating it.

The simplest actor-critic algorithm (QAC)

Aim: Search for an optimal policy by maximizing $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t, \theta_t)$, observe r_{t+1}, s_{t+1}, and then generate a_{t+1} following $\pi(a|s_{t+1}, \theta_t)$.

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \left[r_{t+1} + \gamma q(s_{t+1}, a_{t+1}, w_t) - q(s_t, a_t, w_t) \right] \nabla_w q(s_t, a_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q(s_t, a_t, w_{t+1})$$

Advantage actor-critic (A2C) or TD actor-critic

Aim: Search for an optimal policy by maximizing $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\pi(a|s_t, \theta_t)$ and then observe r_{t+1}, s_{t+1}.

TD error (advantage function):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$$

Actor (policy update)

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t)$$

重要性采样

Off-policy actor-critic based on importance sampling

Initialization: A given behavior policy $\beta(a|s)$. A target policy $\pi(a|s,\theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s,w_0)$ where w_0 is the initial parameter vector.

Aim: Search for an optimal policy by maximizing $J(\theta)$.

At time step t in each episode, do

Generate a_t following $\beta(s_t)$ and then observe r_{t+1}, s_{t+1}.

TD error (advantage function):

$$\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_w v(s_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \delta_t \nabla_\theta \ln \pi(a_t|s_t,\theta_t)$$

Deterministic actor-critic algorithm

Initialization: A given behavior policy $\beta(a|s)$. A deterministic target policy $\mu(s,\theta_0)$ where θ_0 is the initial parameter vector. A value function $v(s,w_0)$ where w_0 is the initial parameter vector.

Aim: Search for an optimal policy by maximizing $J(\theta)$.

At time step t in each episode, do

Generate a_t following β and then observe r_{t+1}, s_{t+1}.

TD error:

$$\delta_t = r_{t+1} + \gamma q(s_{t+1}, \mu(s_{t+1},\theta_t), w_t) - q(s_t, a_t, w_t)$$

Critic (value update):

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w q(s_t, a_t, w_t)$$

Actor (policy update):

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu(s_t,\theta_t) \left(\nabla_a q(s_t, a, w_{t+1})\right)|_{a=\mu(s_t)}$$

知识回顾：

从基于模型的 DP 开始。

走向无模型的 MC 和 TD。

从表格型走向价值近似（DQN）。

从 Value-based 走向 Policy-based（REINFORCE）。

最终融合为 Actor-Critic 架构（QAC, A2C, DPG）。

现代深度强化学习的前沿算法（如 TRPO, PPO, SAC, TD3）基本都是在 Actor-Critic 框架下修修补补（如加 Trust Region、加双重网络、加最大熵等）得来的

https://chat.deepseek.com/share/0i4lufe7zz0k8gj11l

